Zero-Custody Architecture: Naming a Synthesis for Confidential- and Regulated-Data Web Applications

Author: Scott Thomas Stone **Affiliation:** Metropolitan State University, College of Business and Management (COBM) **Contact:** scott.stone@metrostate.edu · ORCID 0000-0003-2718-6848 · GitHub sstone55423
Date: 2026 **License:** Creative Commons Attribution 4.0 International (CC-BY-4.0)

Cite as: Stone, S. T. (2026). *Zero-Custody Architecture: Naming a Synthesis for Confidential- and Regulated-Data Web Applications.* Zenodo. https://doi.org/[DOI-to-be-minted-on-deposit]

Abstract

Web applications that handle confidential or regulated data conventionally place that data in a vendor's custody, making the server both the compliance boundary and the breach surface. This paper names and articulates the **Zero-Custody architecture**: a coherent synthesis in which the server holds no user data at all. Its five defining properties are (1) 100% client-side computation over pure domain functions, (2) zero-knowledge local storage under a passphrase-derived key, (3) zero data custody by any server, (4) an edge-served static single-page application paired with a thin stateless Worker that reuses the *same* pure domain code to expose a machine (MCP) API, and (5) on-device or bring-your-own-key AI as a first-class citizen, with AI forbidden from computing figures. None of the ingredients are new: local-first software, end-to-end-encrypted web apps, browser-direct AI clients, in-browser inference, and edge-served static sites are all established. Following the naming tradition of "JAMstack" and "local-first," the contribution is the synthesis, its articulation, and its application to regulated domains. We ground the pattern in a reference implementation, the FAIR-based QCR Workbench.[1]

---

1. Introduction and motivation

Three forces now collide in ordinary line-of-business software. First, an increasing share of analytical work runs in the browser, delivered as a web application rather than installed software. Second, much of the data that work touches is confidential or regulated: cyber-risk registers naming real assets and exposures, health information, education records, financial detail. Third, generative AI has become an expected feature, and the reflexive way to add it is to route user text through a server the vendor controls.

The conventional architecture answers all three with the same move: put the data on a server. The server authenticates users, stores their records, runs the computation, and brokers the AI calls. This is familiar and it works, but for confidential and regulated data it converts every convenience into a liability that someone else must now manage on the user's behalf.

The liabilities are concrete. **Vendor risk:** an organization evaluating the tool must assess the provider's security program, often through a lengthy questionnaire, a data-processing agreement, and a request for a SOC 2 report, before a single confidential scenario may be entered. **Breach surface:** a server that holds many customers' data is a standing, multi-tenant target; a single compromise exposes everyone at once, and the users bear a loss they did not cause and could not prevent. **Compliance burden:** because the vendor is now a custodian or processor of regulated data, frameworks such as SOC 2, HIPAA, FERPA, and GDPR attach to the vendor, imposing ongoing audit, attestation, and contractual obligations whose cost is ultimately paid by every user. The custody relationship, entered into for convenience, is the thing that generates the burden.

This paper argues that for an important class of tools — single-user, analytical, working over confidential inputs — the custody relationship is avoidable, and that avoiding it dissolves rather than mitigates the liabilities above. There is no vendor security program to assess because the vendor holds nothing; there is no

[1] Live instance and source code available at https://www.qcr-workbench.org/

multi-tenant breach surface because there is no tenancy; there is no service organization to audit because no service organization ever receives the data.

We call the composition that achieves this **Zero-Custody**. It is not a new invention so much as a named, deliberate assembly of five established techniques, chosen because together they close every path by which user data could reach a server. Section 2 credits the prior art in detail. Section 3 defines the pattern and the trade-offs it accepts. Section 4 walks a working reference implementation. Sections 5 and 6 treat the security posture and the boundaries of applicability.

2. Related work and prior art

The Zero-Custody pattern is a synthesis, and intellectual honesty requires stating plainly that each of its ingredients is established work by others. The contribution claimed here is the composition and its naming, in the explicit tradition of how "JAMstack" and "local-first software" named compositions of pre-existing ideas rather than inventing their parts.

Local-first software. The clearest articulation of putting the user's data and computation on the user's own device — while retaining the collaborative virtues of cloud software — is the "local-first software" essay by Kleppmann, Wiggins, van Hardenberg, and McGranaghan at Ink & Switch (2019). Their seven ideals (no spinners, your work is not trapped on one device, the network is optional, and so on) are the direct ancestor of property (1) and much of property (3). Zero-Custody adopts the local-first stance and specializes it for the single-user, confidential-data case, where the collaboration goals that motivate CRDT-based sync are explicitly out of scope.

Zero-knowledge and end-to-end-encrypted web applications. Client-side encryption under a key the server never sees is the operating principle of a family of production tools — Standard Notes, Bitwarden, Proton (Mail, Drive, Pass), and Cryptee among them. These systems demonstrate that a browser can hold a passphrase-derived key in memory and encrypt records before they ever leave the device. Property (2) is a direct application of this technique, differing mainly in that Zero-Custody does not synchronize the ciphertext to a server at all; the encrypted store simply stays local.

Bring-your-own-key, browser-direct AI clients. A growing category of front ends — TypingMind, big-AGI, and LibreChat among them — lets the user supply their own provider API key and calls the model provider directly from the browser, with no intermediary server holding the key or the conversation. Property (5)'s cloud path is this pattern.

In-browser and on-device LLM inference. WebLLM (MLC), Transformers.js (Hugging Face), Chrome's built-in AI (Gemini Nano), and Ollama for local servers have made it practical to run language models without any network call to a provider. Property (5)'s on-device path rests entirely on this body of work.

Edge-served static sites plus serverless functions. The pattern of serving a static front end from a CDN and pairing it with small serverless functions was popularized and named "JAMstack" by Netlify, and is realized here on Cloudflare Workers and Pages. Property (4) uses exactly this shape — with the specific twist that the serverless function is stateless with respect to user data and reuses the front end's own domain code.

Positioned against this prior art, Zero-Custody claims no new cryptography, no new inference engine, and no new deployment primitive. It claims that assembling these five specific properties *together* yields a distinct and useful architecture for confidential- and regulated-data tools, and that giving the assembly a name makes it easier to specify, evaluate, and adopt — the same service "JAMstack" and "local-first" performed for their compositions.

3. The Zero-Custody pattern

Definition (one sentence): A Zero-Custody application computes entirely on the client over pure domain functions, stores its data only in a zero-knowledge, passphrase-encrypted local store, and is served from an edge that holds no user data and runs at most a stateless API reusing the same domain code — so that no server ever receives, computes on, or retains user content, and any AI is on-device or keyed by the user directly to the provider.

3.1 The five defining properties

1. **100% client-side computation.** All domain logic is implemented as pure functions that run in the browser. The server never computes on user data; a total server outage changes no analytical result. Purity is load-bearing: because the functions are deterministic and stateless, they can be executed identically in the page and in an edge function (property 4) and are trivially testable.
2. **Zero-knowledge local storage.** User records are encrypted in the browser under a key derived from the user's passphrase and never persisted; the key exists only in memory while the store is unlocked. The storage medium (here, IndexedDB) holds ciphertext only. A party with full read access to the device's storage, but not the passphrase, learns nothing.
3. **Zero data custody by any server.** No backend holds user data. The edge serves static assets and, optionally, a stateless API that computes-and-returns without persisting. There is no user table, no document store of user content, no per-user server state to breach or to subpoena.
4. **Edge-served static SPA plus a thin, domain-reusing Worker.** The application ships as a static single-page app served from the edge. A thin Worker on the same origin may expose a machine-callable API — here, a Model Context Protocol (MCP) endpoint — that **reuses the exact same pure domain code** the browser runs. Humans and autonomous agents therefore hit byte-identical logic, and the machine API inherits the front end's correctness guarantees for free.
5. **On-device / bring-your-own-key AI as first-class.** AI assistance, when present, runs on-device (WebLLM over WebGPU, Chrome built-in AI, local Ollama) or is called browser-to-provider-directly with the user's own key — never proxied through a vendor server. AI is strictly forbidden from computing numbers; every figure comes from the deterministic domain functions, and AI is confined to prose around figures it is handed.

All five hold together, or the pattern is not Zero-Custody. Drop (1) and the server computes on plaintext. Drop (2) and local data leaks at rest. Drop (3) and custody returns. Drop the reuse in (4) and the machine API becomes a second, drifting implementation. Relax (5) and user content flows through a proxy.

3.2 Forces the pattern resolves

Zero-Custody resolves the tension identified in Section 1 between *browser-delivered convenience* and *confidential-data safety*. It removes vendor risk by removing the vendor's custody; it removes the multi-tenant breach surface by removing tenancy; it removes the vendor's compliance burden by removing the vendor's role as custodian or processor. It also resolves a subtler tension — between *serving a machine API* and *maintaining a single source of truth for the math* — by having the API reuse the client's own domain functions rather than reimplementing them.

3.3 Forces the pattern imposes

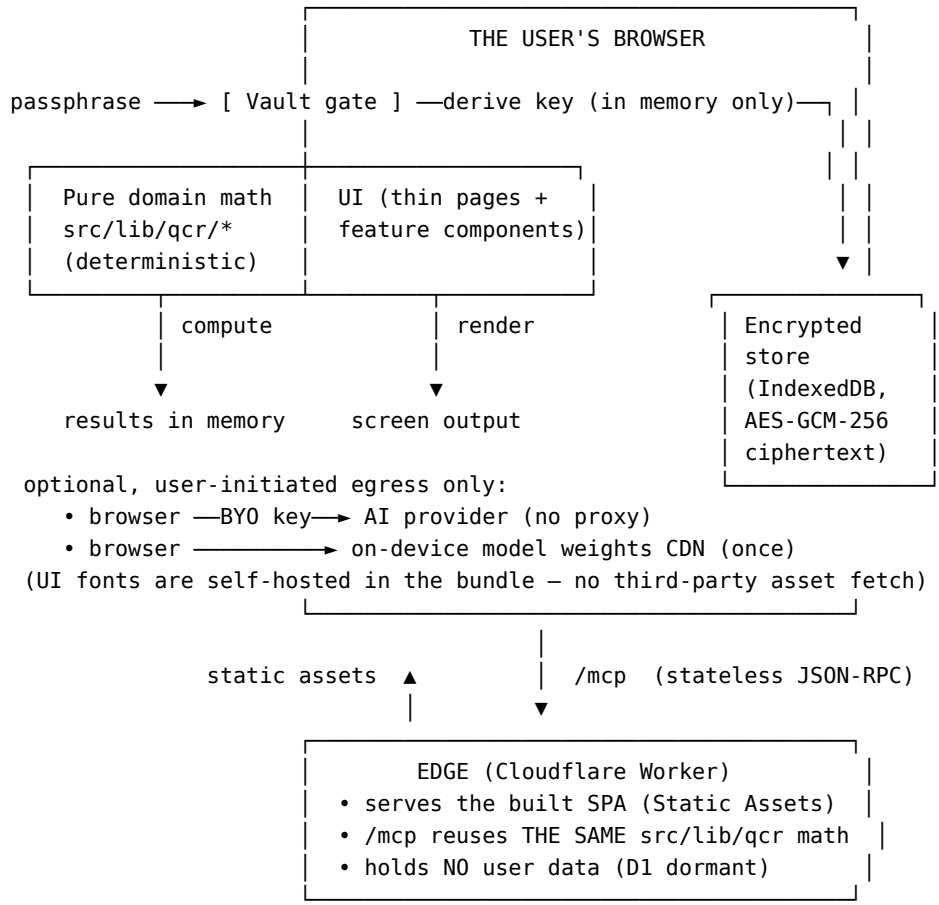
The pattern is not free. Because the passphrase *is* the entire security boundary and no party holds a copy of the key, **a forgotten passphrase is unrecoverable** — there is no reset, no recovery email, no vendor help desk. **Durability becomes the user's responsibility:** with no server copy, loss or wipe of the device loses the data unless the user has made a backup, so the design must make encrypted backup easy and obvious. **Cross-device sync is not automatic:** getting data to a second device requires either a manual encrypted backup transfer or an added end-to-end-encrypted sync layer, which reintroduces engineering (though not necessarily custody). And enabling on-device model downloads and browser-direct AI calls forces a **broad `connect-src` Content-Security-Policy**, because model weights and BYO-key providers live on arbitrary hosts; that directive consequently cannot serve as an exfiltration barrier, and the anti-XSS defense must rest on `script-src` instead. These are real costs, accepted deliberately in exchange for eliminating custody.

4. Reference architecture: QCR Workbench

QCR Workbench is a local-first, single-user workbench for quantitative cyber-risk analysis built on the FAIR model (Factor Analysis of Information Risk). It turns a vaguely worded risk into a money-denominated

estimate — an annualized loss expectancy, a full loss distribution with tail percentiles, and a cost-justified comparison of candidate controls. It is a faithful instance of all five Zero-Custody properties and is used here as the worked example.

4.1 Layered data-flow (text diagram)



The vertical boundary matters: everything above the edge box runs on the user's machine; the edge below serves and (statelessly) computes, but never stores or receives risk data.

4.2 The zero-knowledge vault (property 2)

The vault is implemented in `src/lib/localdb/crypto.js` and `src/lib/localdb/store.js`. The exact parameters are:

- **Key derivation:** PBKDF2 with **SHA-256** over a **random 16-byte salt**, producing a **256-bit AES-GCM key**. New vaults derive at **600,000 iterations**, matching current OWASP guidance for PBKDF2-HMAC-SHA256; each vault records its own iteration count in its metadata, and vaults created before this change continue to unlock at their original 250,000 (`LEGACY_ITERATIONS`). Storing the work factor alongside the salt makes the parameter versioned rather than compiled in, so it can be raised again without stranding existing data. WebCrypto exposes no native Argon2id, which OWASP prefers outright, so PBKDF2 is a platform constraint of the browser-vault approach and is disclosed as such. The passphrase is imported as non-extractable key material; the derived key is created non-extractable as well (`deriveKey`, `crypto.js`).
- **Encryption: AES-GCM-256**, authenticated, with a **fresh random 12-byte IV per record** (`encryptJSON`). Because GCM is authenticated, tampering with stored ciphertext is detectable on decrypt.

- **Key lifetime:** the derived key is held in a module-level variable and **never persisted**. `lockVault()` sets it to `null` and clears the decrypted-record cache; a manual lock, the configurable auto-lock timer, and a full page reload all discard it. Consequently, a page reload re-locks the vault by design.
- **Unlock verification:** the vault stores a small encrypted check-token; `unlockVault()` derives a candidate key from the entered passphrase and the stored salt, attempts to decrypt the token, and accepts the passphrase only if it matches — so a wrong passphrase fails cleanly without a plaintext oracle.
- **Multi-store isolation:** each workspace ("library") is a *separate* encrypted IndexedDB database with its own salt and passphrase; a non-secret registry in `localStorage` lists their display names for the lock-screen picker and holds **no passphrases and no risk data**.
- **Secrets stay in the vault:** optional AI-provider API keys live in the encrypted `AppSettings` record inside the vault, never in `localStorage` or plaintext. Only genuinely non-secret material (theme, language, auto-lock minutes, the store registry, and an opt-in lock-screen email) lives unencrypted, because it must be readable before the vault unlocks.

`createVault()` refuses to overwrite an existing vault, preventing a second browser tab from replacing the salt and orphaning every existing record — a data-integrity guard rather than a confidentiality one.

4.3 The pure domain-math layer (property 1)

All risk math lives in `src/lib/qcr/` as pure functions: FAIR decomposition and deterministic expected loss (`fair.js`), modified-PERT sampling and seeded Monte Carlo simulation with exceedance curves (`simulation.js`), treatment economics and budget optimization (`treatments.js`), tornado sensitivity (`sensitivity.js`), and calibration (`calibration.js`). Determinism is enforced and tested: the Monte Carlo simulation is seeded and reproducible, so an instructor's reference figures reproduce exactly on every machine (for the documented `ransomware` sample: deterministic ALE **$611,274**; simulated mean at 20,000 iterations, seed 42, **≈ $802k**; a default treatment yielding residual **$276,000**, net benefit **$185,275**, return on control **124%**). The raw simulation loss array is never persisted — only the summary, histogram bins, and exceedance points — because it is recomputable from the seed; treatment results are never stored at all but recomputed from the current model at render, so they can never disagree with the estimates. These invariants are what make the same code safe to run on the edge.

4.4 The edge shell that reuses domain code (property 4)

The deployment wraps the static SPA in a single Cloudflare Worker (`worker/index.js`, configured by `wrangler.jsonc`). The Worker serves the built app via Static Assets with single-page-application fallback, redirects `www` to the apex so there is one canonical origin, and exposes a **public, stateless MCP endpoint** at `/mcp` (`worker/mcp.js`). The decisive detail is in `wrangler.jsonc`: the alias `"@": "./src"` lets the Worker bundle import the SPA's own modules, and the MCP tool registry (`src/lib/mcp/tools.js`) is explicitly "the single source of truth shared by the remote MCP server and the in-page WebMCP provider," wrapping the *same* `src/lib/qcr` functions. An external agent calling `tools/call` over JSON-RPC therefore executes byte-identical math to what the human sees in the browser. This is also, as the code comments note, the strong form of the AI-governance rule "AI never computes numbers": an agent obtains real, audited figures by *calling* the trusted math rather than guessing it.

Crucially, the Worker holds no user data. A D1 database is wired in configuration but **dormant by explicit choice** — the reference deployment persists no risk data server-side; the `/mcp` endpoint computes and returns, persisting nothing. An optional `/api/ai` route offers a free, keyless AI provider via Workers AI, but even there governance is unchanged: the prompt arrives with the figures already computed, and the model only writes prose around them.

4.5 AI governance (property 5)

AI is optional and boxed in (`AI-GOVERNANCE.md`). Under NIST AI RMF, ISO/IEC 42001, and the EU AI Act, the app is a *deployer* of third-party general-purpose models, not a provider: it ships no model and the user credentials every model used. Four rules hold in code, not merely in policy: **AI never does the math** (every figure comes from `src/lib/qcr/`; prompts embed computed figures and instruct the model not to recompute

them); **human-in-the-loop** (AI-suggested assumptions and treatments are staged and enter the model only on explicit per-item acceptance); **provenance and audit** (each AI output is labeled with provider, model, and timestamp, written to the audit log, and stamped into the exported report, with an inputs hash driving staleness detection); and **privacy by architecture** (cloud calls go browser-to-provider-directly with the user's own key — `src/lib/anthropic.js` sets `anthropic-dangerous-direct-browser-access`, and other providers are called directly from `src/lib/ai.js` — while WebLLM, Chrome built-in AI, and Ollama keep all text on-device).

5. Security and privacy analysis

Threat model. The design is local-first and single-user. The assets to protect are the vault contents (confidential scenarios and estimates) and the API keys while the vault is unlocked. There is no multi-tenant server holding user data, so the class of attack that dominates conventional web-app risk — compromise of a central datastore — has no target here.

What an attacker who compromises the host or edge cannot get. Because the edge serves static assets and a stateless API, an adversary who fully compromises the hosting infrastructure — the CDN, the Worker, the account — obtains the application *code* and can serve a malicious build to future visitors, but obtains **no existing user data**, because none is stored there. There is no user database to exfiltrate, no document store to dump, no session store of plaintext. This is a categorical reduction, not a mitigation: the confidential corpus that a conventional breach would expose does not exist on the server to begin with. (The residual risk — a poisoned future build — is real and is *bounded*, not eliminated, by the static-only, no-serve-time-injection posture; it is a supply-chain/integrity risk rather than a bulk-data-exfiltration risk, and it is treated in full immediately below.)

The poisoned-build residual. The one risk this pattern does not eliminate deserves to be named plainly rather than left in a parenthetical. Because the browser refetches the application on every visit, an adversary who controls the origin can serve a modified build that captures the passphrase or the plaintext at the moment the user unlocks. Zero custody bounds the blast radius — such an attacker obtains only what users enter *after* the compromise, never the accumulated corpus, and never anything from users who do not return — but it does not prevent the attack. Integrity is the one property a local-first design cannot supply for itself, because the code still arrives from somewhere.

The standard remedies are well established, and this deployment implements them only partially. The strict `script-src` CSP with per-build hashes constrains what an *injected* script may do, but offers nothing when the build itself is the adversary, since a poisoned build ships its own valid hashes. Subresource Integrity is the weakest of the three here: it guards same-origin bundles against a CDN that diverges from the entry document, but an attacker who controls the origin controls `index.html` too and simply updates both. What would materially close the loop is **reproducible builds**, so any third party can rebuild the published artifact from the tagged source and compare digests, paired with **signed release artifacts and provenance attestation** (Sigstore/cosign with SLSA-style provenance), so that the bundle a user receives is verifiably the one the maintainer produced from a known commit. Neither is implemented at the time of writing. Both are roadmap items, and their absence is the honest boundary of this section's claims.

Stated generally: zero custody removes the server as a *data* trust anchor but leaves it intact as a *code* trust anchor. Closing that remaining gap is a supply-chain engineering problem rather than an architectural one, and it is the subject of follow-on work.

What an attacker with the device's storage cannot get. IndexedDB holds only AES-GCM ciphertext under a key derived from a passphrase that is never stored. Absent the passphrase, the encrypted records and the API keys in `AppSettings` are opaque; AES-GCM's authentication also makes tampering detectable.

XSS and injection. The production build ships a CSP (`script-src 'self'` plus per-build sha256 hashes, `object-src 'none'`, `frame-src 'none'`, `base-uri 'self'`, `form-action 'self'`), which blocks execution of injected or inline scripts — the real anti-XSS control. AI-generated and report content is rendered with `react-markdown` *without* `rehype-raw`, so any HTML in generated text is escaped rather than executed, and there are no `dangerouslySetInnerHTML` sinks anywhere. The `connect-src` directive is deliberately broad

(`'self' https: http://localhost:*`) to permit BYO-key providers, model-weight CDNs, and local Ollama; it is therefore explicitly *not* an exfiltration barrier, and the design says so — the anti-XSS defense rests on `script-src`, not on `connect-src`.

Regulatory posture — no service organization to audit. The honest regulatory answers follow directly from the absence of custody, and none is a certification; they are architectural statements verifiable from source. **SOC 2** attests that a *service organization* safeguards customer data it holds; QCR Workbench holds nothing, so there is no organization to audit (the static-file host maintains its own infrastructure attestations, which is a separate matter). **HIPAA** contemplates a business associate that receives PHI; because no vendor ever receives user data, there is no business-associate relationship with the app itself — the one caveat being that a user who enables *cloud* AI transmits scenario text to their own chosen provider and owns that relationship, or avoids it entirely with on-device AI. **FERPA** has the same shape for education records. **GDPR** finds the app in neither controller nor processor role over user content: no telemetry, no tracking identifiers, processing only on the user's device under the user's control; a user exporting to a cloud AI provider is choosing their own processor. The single, user-initiated, user-configured off-device path — the optional AI call — is the entirety of the egress question, and it is the user's to govern.

6. Limitations and when *not* to use Zero-Custody

Zero-Custody is a specialized pattern, not a default. It is a poor fit — or an outright wrong one — in several common situations.

- **Collaborative, multi-user data.** When many users must read and write shared records concurrently, the local-first, single-user vault does not apply as-is. Real-time collaboration needs a synchronization layer; achieving it without reintroducing custody requires end-to-end-encrypted sync (e.g., CRDT-based), which is substantial added engineering and outside this pattern's minimal form.
- **Server-authoritative workflows.** Any process that must enforce rules the client cannot be trusted to enforce — payment capture, inventory decrement, authoritative timestamps, regulatory submission of record — needs a trusted server that holds and validates state. Zero-Custody deliberately has no such authority.
- **Data an organization must be able to recover.** When an employer, institution, or custodian must retain or recover a user's data independent of that user — for legal hold, offboarding, continuity, or audit — the property that "a forgotten passphrase is unrecoverable and no vendor can help" becomes a defect, not a feature. Such settings need escrow or organizational custody by design.
- **Fleet-managed durability and device loss.** Where the organization, not the individual, is accountable for the data surviving a lost or wiped device, pushing durability onto each user is inappropriate. Zero-Custody puts backup in the user's hands; some contexts cannot accept that.
- **Cross-device continuity as a hard requirement.** If seamless multi-device access is non-negotiable, the manual-backup transfer the minimal pattern offers will disappoint, and an E2E-sync layer must be added before the pattern is viable.

The honest positioning: Zero-Custody is excellent for **single-user analytical tools over confidential inputs**, and progressively less suitable as requirements shift toward shared state, server authority, and organizational recoverability. Naming the pattern is partly meant to make this boundary easy to reason about before adopting it.

7. Conclusion and a call for adoption

The dominant reason confidential- and regulated-data web tools are slow to build, slow to procure, and heavy to operate is custody: the server holds the data, and holding it generates vendor risk, breach surface, and compliance burden in proportion to its sensitivity. For an important class of applications, that custody is not required by the problem — it is an artifact of a default architecture. When the domain math is expressible as pure functions and the workflow is single-user and analytical, the data can stay on the user's device, encrypted under a key only the user holds, while the edge does nothing but serve assets and, optionally, run the *same* math statelessly for machine callers.

We have named this composition **Zero-Custody** and stated it as five properties that must hold together, and we have shown a faithful, working instance in the QCR Workbench, down to its exact cryptographic parameters. We claim no new primitive; every ingredient is credited to those who built it. What a name does — as "JAMstack" and "local-first" did before it — is let practitioners specify an intent in a word, evaluate a candidate design against a checklist, and choose the pattern where it fits and reject it where it does not.

The call is modest and specific: when you are next asked to build a browser tool over confidential or regulated data, ask whether the server needs to hold anything at all. Often it does not. Where it does not, Zero-Custody offers a way to ship the tool with no custody to defend, no service organization to audit, and no bulk-data breach to suffer — and to say so in terms a security reviewer can verify from the source.

References

- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). *Local-first software: You own your data, in spite of the cloud.* Ink & Switch. https://www.inkandswitch.com/local-first/
- Ink & Switch. (2019). *Local-first software* (project page). https://www.inkandswitch.com/local-first/
- Standard Notes. *End-to-end encrypted notes* (client-side encryption architecture). https://standardnotes.com/
- Bitwarden. *Security and encryption whitepaper* (zero-knowledge, PBKDF2/Argon2 key derivation). https://bitwarden.com/help/bitwarden-security-white-paper/
- Proton AG. *Proton security and end-to-end encryption model.* https://proton.me/
- Cryptee. *Zero-knowledge, client-side-encrypted documents and photos.* https://crypt.ee/
- MLC AI. *WebLLM: In-browser LLM inference via WebGPU.* https://github.com/mlc-ai/web-llm
- Hugging Face. *Transformers.js: State-of-the-art machine learning in the browser.* https://huggingface.co/docs/transformers.js
- Google / Chrome. *Built-in AI (Gemini Nano) — on-device model APIs.* https://developer.chrome.com/docs/ai/built-in
- Ollama. *Run large language models locally.* https://ollama.com/
- TypingMind. *Bring-your-own-key browser-direct AI front end.* https://www.typingmind.com/
- big-AGI. *Browser-direct, BYO-key generative-AI application.* https://big-agi.com/
- LibreChat. *Self-hostable, BYO-key AI chat interface.* https://www.librechat.ai/
- Netlify. *JAMstack: Modern web architecture (static front end + serverless functions).* https://jamstack.org/
- Cloudflare. *Workers and Pages — edge-served static assets and serverless functions.* https://developers.cloudflare.com/workers/
- Anthropic. *Model Context Protocol (MCP) specification.* https://modelcontextprotocol.io/
- Stone, S. T. (2026). *Measuring Cyber Risk: A Practical Quantitative Approach to Security Risk Analysis.* Stjarna Press, Minneapolis, MN. ISBN 979-8-9968997-2-2 (paperback), ISBN 979-8-9968997-7-7 (e-book).
- Open Group / FAIR Institute. *Factor Analysis of Information Risk (FAIR) — an open standard for quantifying information risk.* https://www.fairinstitute.org/
- Stone, S. T. (2026). *QCR Workbench: A local-first quantitative cyber risk workbench* (software). https://github.com/sstone55423/qcr-workbench